# Evaluating Rule Compliance and Decision Traceability in a Social Assistance Eligibility Prototype

Dzaky Ahsanal Khuluq and Annisa Nailatul Izzah

Electronic Engineering Polytechnic Institute of Surabaya, Surabaya, Indonesia
dzakyahsanal@it.student.pens.ac.id, annisanaiizah24@it.student.pens.ac.id
https://it.pens.ac.id/

**Abstract.** The distribution of social assistance relies on consistent and traceable citizen eligibility assessments, whereas manual or decentralized examinations risk producing ununiform and uncorrectable evaluations. This study evaluates the Sociovault prototype, a Flask-based web application running a three-stage rule engine for a single assistance program: data completeness checks, four-criterion evaluations, and final officer confirmation. This research aims to test whether the system's recommendations and rationales follow defined rules, and whether decisions can be traced and corrected. The method used is rule compliance testing against a rule matrix with 60 synthetic cases (Test 1) along with boundary value analysis and duplicate verification (Test 2). The results show label compliance of 100% and rationale compliance of 100% across all evaluated deterministic scenarios. Furthermore, the evaluation identifies an architectural boundary: while citizen socioeconomic attributes can be updated via an administrative appeal workflow, the engine enforces strict single-verification immutability, requiring formal re-evaluation mechanisms for updated profiles.

**Keywords:** Sociovault · social assistance · eligibility verification · rule engine · decision traceability · rule compliance

## 1 Introduction

Social assistance programs serve as an essential government mechanism to protect vulnerable populations and alleviate poverty [8]. In developing countries, including Indonesia, targeting welfare recipients remains a persistent challenge characterized by exclusion errors (ineligible households receiving aid) and inclusion errors (impoverished households excluded from benefits) [1,3]. Under the national Electronic-Based Government System framework (*Sistem Pemerintahan Berbasis Elektronik* / SPBE) [9], public service delivery must be transformed into integrated, objective, and auditable digital processes. Examining applicant data is therefore a vital operational step to ensure that eligibility recommendations are grounded in verifiable household parameters and predefined regulatory criteria rather than discretionary manual judgments.

Several systemic risks must be addressed during automated eligibility assessments, including incomplete applicant records, duplicate claims across overlapping assistance programs, and determinations delivered without clear, verifiable explanations [17,10]. Incomplete records frequently cause automated engines to either silently drop applicants or issue erroneous rejections, whereas duplicate records distort budgetary allocation and violate fairness principles. Crucially, algorithmic decisions rendered without granular, criteria-level rationales hinder caseworkers and citizens from understanding why assistance was denied and prevent the timely correction of erroneous data. In public assistance administration, accountability requires that every decision can be audited against its underlying policy rules [15,16].

A viable technological approach is rule-based eligibility evaluation, which formalizes policy regulations into explicit, deterministic criteria [18]. To maintain operational integrity, rule engines must be supported by decision traceability—the ability to reconstruct the exact relationship between applicant input parameters, applied threshold criteria, and generated recommendations. Traceability requires maintaining comprehensive audit logs of all inspection steps, recording parameter versions, and systematically isolating incomplete or ambiguous cases for human inspection (`perlu_ditinjau`) rather than prematurely rejecting vulnerable citizens without explicit evidentiary cause.

To investigate these operational requirements, this study evaluates rule compliance and decision traceability in a social assistance eligibility prototype called **Sociovault**. Specifically, this research addresses two primary Research Questions (RQs):

- **RQ1:** *To what extent does the prototype's multi-stage rule engine adhere to defined program eligibility criteria and boundary conditions without making premature automated decisions on incomplete or anomalous applications?*
- **RQ2:** *Can the decision rationale generated by the system be traced and audited at the per-criterion level, and how does the prototype handle data correction and re-evaluation when applicant profiles are updated?*

The evaluation examines 60 synthetic cases across standard and edge-case profiles (Test 1), boundary threshold values, and duplicate submission scenarios (Test 2). The findings demonstrate deterministic rule and rationale compliance while uncovering critical architectural constraints regarding post-decision immutability and citizen data re-evaluation.

## 2 Related Work

### 2.1 Rule-Based Eligibility and Transforming Policies into Decision Models

Research on social service eligibility automation emphasizes that eligibility rules are often embedded in complex policy documents, making them difficult to translate into executable logic. Wei et al. proposed a human-AI collaborative approach

using natural language processing and knowledge graphs to transform eligibility policy documents into executable and interpretable decision models based on Decision Model and Notation (DMN) standards and Friendly Enough Expression Language (FEEL) [18]. This approach separates regulatory policy definitions from core application logic.

Industry systems similarly advocate decoupling business rules into auditable decision engines to ensure regulatory audit readiness [11]. In contrast, the Sociovault prototype adopts a lightweight architecture tailored for municipal administration: criteria parameters are stored in relational database tables (`help_program`) and evaluated by a deterministic rule engine (`run_verify` / `jalankan_verifikasi` in Python). Consequently, verifying that this embedded engine strictly adheres to coded specifications is essential.

### 2.2 Explainable and Auditable Decision Support for Public Services

The literature on automated public assistance systems increasingly prioritizes auditability over generic model interpretability. Sunny demonstrated that explanations produced by automated eligibility systems often diverge from underlying legal statutes, proposing a neuro-symbolic framework that ties generated explanations to formal requirements ontologies and satisfiability solvers [15]. Related work reinforces that public-sector algorithmic explanations must directly cite governing criteria so that determinations remain contestable by administrative officers and applicants [16].

While those studies investigate high-level legal argumentation for programs such as CalFresh in the United States, Sociovault operates at the operational administrative tier: its output is a structured, per-criterion evaluation detailing applicant values, comparative operators ($\leq, \geq$), program thresholds, and individual pass/fail statuses. The primary evaluation concern for Sociovault is therefore whether these structured details faithfully represent input data and applied rules.

### 2.3 Deduplication and Recipient Identity Matching

Duplicate benefit disbursement has long presented an administrative vulnerability in public welfare. A landmark U.S. General Accounting Office (GAO) audit revealed that cross-jurisdictional duplicate welfare payments persisted primarily due to missing or unverified identification numbers and fragmented data formats [17]. Historically, social service agencies implemented rule-based cross-matching: early state circulars employed exact name and birthdate matching to identify potential duplicate recipients for human case review [13,14].

Modern social protection architectures increasingly deploy automated entity resolution systems. The OpenG2P deduplication framework integrates multi-attribute fuzzy matching with biometric verifications to resolve duplicates across disparate identity sources [4]. The openIMIS initiative designed dedicated deduplication modules incorporating probabilistic similarity algorithms and persistent audit logs [5]. Similar multi-tiered deduplication protocols have been codi-

fied in regional social welfare guidelines [2], while system harmonization remains critical for humanitarian cash transfer programs [6].

In its current prototype version, Sociovault implements deterministic deduplication through unique constraints on National Identity Numbers (NIK) and unique citizen–program verification pairs. Rather than conducting probabilistic entity resolution, Sociovault enforces strict single-entry registration, making evaluation of its boundary behavior and error rejection critical.

### 2.4 Identity Data Quality and Handling of Missing Data

Incomplete or erroneous identity data is a primary source of targeting misclassification in public welfare registries [17]. Established best practices in public assistance decision systems emphasize that missing mandatory attributes should be treated as pending cases requiring investigation rather than being summarily categorized as ineligible, preventing the unfair exclusion of entitled citizens. Sociovault aligns with this principle by routing missing mandatory fields and zero-income anomalies to a provisional "`perlu_ditinjau`" (needs review) status rather than generating an automated negative determination.

### 2.5 Sociovault's Position Relative to Previous Studies

Table 1 summarizes the focus of Sociovault in comparison to related literature. Existing work has primarily addressed high-level policy translation or biometric deduplication in large-scale infrastructures. Sociovault evaluates the practical integration of multi-stage deterministic rule compliance, granular criterion traceability, and post-verification immutability within a web-accessible prototype.

## 3 Methodology

### 3.1 Assistance Program Criteria and Prototype Setup

The Sociovault prototype is implemented using Python with the Flask web framework [7] and SQLAlchemy ORM [12]. The prototype database contains predefined assistance programs initialized via `seed.py`. For this evaluation, Program A was selected as the representative simulation benchmark. The criteria and thresholds configured for Program A are detailed in Table 2.

### 3.2 Verification Status Workflow

Eligibility determination follows a two-tier verification workflow: automated algorithmic evaluation by the rule engine followed by final administrative confirmation by an authorized officer. Fig. 1 depicts this lifecycle as implemented in `app/verifikasi/engine.py`.

**Table 1.** Comparison of Sociovault with related literature and industry sources

| Source | Main Focus | Identity Matching | Decision Traceability | Measurable Testing |
|---|---|---|---|---|
| Wei et al. [18] | Policy-to-decision model | Not discussed | Objective framing | Qualitative modeling |
| Sunny [15] | Explanation vs law | Not discussed | Legal basis ontology | Case evaluation |
| GAO [17] | Beneficiary deduplication | SSN match | Audit log inspection | Audit analysis |
| California [13,14] | Name & DOB matching | Exact attribute matching | Procedural documentation | Administrative review |
| OpenG2P [4] | Entity resolution | Fuzzy & biometric matching | Not discussed | Platform specification |
| openIMIS [5] | Deduplication system | Fuzzy matching | Audit logging | Requirements spec |
| Sociovault | Prototype eligibility verification | Unique NIK, single verification constraint | Per-criterion audit records, officer notes | Empirical synthetic tests (Section 4) |

**Table 2.** Criteria of the benchmark assistance program (Program A)

| Criteria | Threshold | Rule Operator |
|---|---|---|
| Monthly Income | $\leqslant$ Rp 2,000,000 | Less than or equal |
| Dependent Count | $\geqslant$ 2 | Greater than or equal |
| Total Asset Value | $\leqslant$ Rp 15,000,000 | Less than or equal |
| Welfare Decile | $\leqslant$ 3 | Less than or equal |

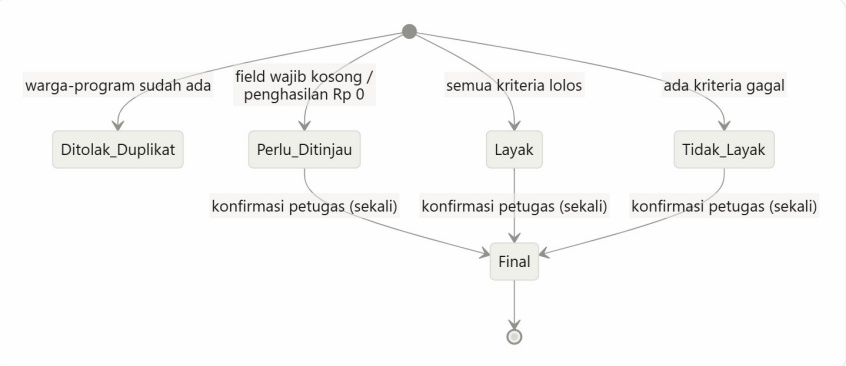



**Fig. 1.** Verification status workflow in Sociovault.

**Table 3.** Verification workflow rules and access control matrix

| Initial State | Action | Allowed Role | System Outcome | Description |
|---|---|---|---|---|
| Unverified | Execute verification | Admin, Officer | `layak`, `tidak_layak`, or `perlu_ditinjau` | Computed by rule engine |
| Unverified | Execute verification | Citizen | Access Denied | Enforced via `role_required` |
| Verified (`layak`) | Re-run verification | Anyone | Rejected (`AlreadyVerifiedError`) | Duplicate check blocks re-entry |
| Verified (`tidak_layak`) | Re-run verification | Anyone | Rejected (`AlreadyVerifiedError`) | Prior record exists |
| Inactive program | Execute verification | Anyone | Rejected (`ValueError`) | Program deactivated |
| Pending final | Final confirmation | Admin, Officer | Committed final status | Mandatory audit notes |
| Finalized | Re-confirm | Admin, Officer | Rejected | Single confirmation constraint |

### 3.3 Workflow Rules and Access Control Matrix

System transitions and authorization boundaries are governed by a strict state matrix, as outlined in Table 3.

### 3.4 Rule Engine Execution and Rationale Logging

The verification engine (`jalankan_verifikasi`) executes three sequential stages:

1. **Data Completeness Check:** If any required demographic or socioeconomic field is empty (`None` or blank string), the system halts standard assessment and assigns the status `perlu_ditinjau`, appending the list of missing attributes to the rationale.
2. **Economic Anomaly Detection:** If the reported income is exactly Rp 0, the engine flags the record as `perlu_ditinjau`, generating an explicit directive that a field survey is required to verify real living conditions.
3. **Deterministic Criteria Assessment:** The four program criteria are evaluated: income $\leq$ limit, dependents $\geq$ limit, assets $\leq$ limit, and decile $\leq$ limit. If all criteria are satisfied, the engine assigns `layak`; if one or more criteria fail, it assigns `tidak_layak`.

Every execution produces a structured `detail_pengecekan` dictionary storing the applicant's input value, the program threshold, the evaluated comparison operator, and the pass/fail determination for each criterion.

### 3.5 Heuristic Welfare Decile Estimation

When an applicant lacks a pre-assigned government decile, Sociovault estimates their decile using a baseline formula in `engine.py`: per-capita income is computed as Income/(Dependents+1) and mapped across ten income brackets starting at Rp 600,000. This baseline is adjusted based on asset indicators (housing construction, electricity wattage, and vehicle ownership). In accordance with policy integrity, any pre-existing official decile recorded in the citizen profile strictly overrides the automated heuristic calculation.

### 3.6 Duplicate Prevention and Data Correction Mechanism

Deduplication operates across three architectural layers: (1) a unique database index on the citizen's NIK in the `warga` table, (2) a composite unique constraint on (`id_warga`, `id_program`) in the `verifikasi` table, and (3) an active precheck in the verification engine that queries existing verification rows prior to processing.

### 3.7 Evaluation Methodology and Synthesized Requirements

To rigorously assess the prototype without compromising private citizen data, the evaluation utilizes synthetic cases with fully specified ground-truth conditions. The requirements evaluate deterministic compliance against coded program specifications rather than external policy validity.

## 4 Experimental Design and Results

### 4.1 Test 1: Rule Compliance Evaluation

**Dataset Composition:** Test 1 evaluates 60 synthetic cases with distinct synthetic NIKs, structured into three balanced ground-truth reference groups established prior to system execution, as detailed in Table 4.

**Table 4.** Composition of synthetic cases for Test 1 ($N = 60$)

| Reference Category | Cases ($N$) | Ground-Truth Specification / Profile Characteristics |
|---|---|---|
| Eligible (`layak`) | 20 | Meets all 4 criteria (e.g., income Rp 1.5M, dependents 3, assets Rp 5M, decile 2). |
| Ineligible (`tidak_layak`) | 20 | 15 single-criterion failures (distributed across income, dependents, assets, decile) and 5 multi-criteria failures ($\geq 2$ failing criteria). |
| Needs Review (`perlu_ditinjau`) | 20 | Inconclusive cases: 10 cases with missing mandatory fields (NULL) and 10 anomaly cases with Rp 0 income. |

**Execution Procedure:** Each synthetic case was passed through `jalankan_verifikasi`. The resulting `status_otomatis`, summary text, and `detail_pengecekan` were captured and compared against reference expectations. A determination is labeled compliant if the status matches the reference category. The rationale is compliant if `detail_pengecekan` accurately logs the specific passing or failing criteria.

**Evaluation Metrics and Findings:** The evaluation measures label compliance, rationale compliance, false acceptances, false rejections, and premature automated decisions on review cases. The results are summarized in Table 5.

**Table 5.** Test 1 rule compliance evaluation results ($N = 60$)

| Evaluation Metric | Compliant | Total ($N$) | Rate (%) |
|---|---|---|---|
| Label Compliance (Matching Status) | 60 | 60 | 100.0% |
| Rationale Compliance (Matching Reasons) | 60 | 60 | 100.0% |
| False Acceptance (Ref: `tidak_layak`, Sys: `layak`) | 0 | 20 | 0.0% |
| False Rejection (Ref: `layak`, Sys: `tidak_layak`) | 0 | 20 | 0.0% |
| Premature Automated Decisions on Review Cases | 0 | 20 | 0.0% |

As shown in Table 5, the deterministic rule engine achieved 100.0% label compliance and 100.0% rationale compliance across all 60 cases. In all 20 review cases, the engine correctly avoided making premature eligibility decisions, routing missing data and zero-income anomalies to `perlu_ditinjau`.

### 4.2 Test 2: Boundary Value Analysis and Duplicate Verification

Test 2 evaluates the precision of relational operators at exact threshold boundaries across all four criteria (12 boundary points) and examines duplicate verification prevention.

**Table 6.** Boundary value test points and evaluation outcomes

| Program Criteria | Below Limit | Exact Limit | Above Limit | Engine Outcome |
|---|---|---|---|---|
| Income ($\leqslant$ Rp 2,000,000) | 1,999,999 (Pass) | 2,000,000 (Pass) | 2,000,001 (Fail) | Inclusive ($\leq$) verified |
| Dependents ($\geqslant 2$) | 1 (Fail) | 2 (Pass) | 3 (Pass) | Inclusive ($\geq$) verified |
| Assets ($\leqslant$ Rp 15,000,000) | 14,999,999 (Pass) | 15,000,000 (Pass) | 15,000,001 (Fail) | Inclusive ($\leq$) verified |
| Decile ($\leqslant 3$) | 2 (Pass) | 3 (Pass) | 4 (Fail) | Inclusive ($\leq$) verified |

**Duplicate Submission and Post-Correction Findings:**

1. *Initial Verification:* Running verification for an unverified citizen–program pair executes normally, creating a single record.

2. *Duplicate Verification Attempt:* Re-executing verification for the identical citizen and program is immediately rejected with `AlreadyVerifiedError`. Database row counts remain strictly unchanged.
3. *Post-Correction Re-evaluation (TC-DUP-02):* When an applicant's demographic profile is edited in the citizen registry (e.g., income updated from Rp 2,500,000 to Rp 1,500,000), subsequent verification attempts in `engine.py` are still rejected by the immutable duplicate barrier (`AlreadyVerifiedError`). This indicates that while the database correctly updates the citizen profile, the verification engine enforces strict immutability on prior program decisions, lacking a coupled re-evaluation or revocation workflow.

### 4.3 Test 3: Officer Decision Traceability Protocol Design

To assess whether human officers can effectively interpret the criteria-level rationales generated in `detail_pengecekan`, a structured 10-case evaluation protocol was designed. Under this protocol, officers are presented with verification outputs and asked to identify: (1) which specific criteria caused an applicant's rejection or review status, and (2) what corrective data attributes must be verified. While this protocol establishes a standardized methodology for human-in-the-loop auditability, empirical testing with active administrative officers was not conducted within this synthetic prototype phase and is reserved for future field deployments.

### 4.4 Synthesis of Experimental Outputs

Tables 7, 8, 9, and 10 provide the consolidated experimental results, linking dataset specifications, condition metrics, root-cause analyses, and research questions.

**Table 7.** Consolidated Output (1): Dataset specifications across evaluation suites

| Test Suite | Evaluation Condition | Cases ($N$) | Ground-Truth Specification |
|---|---|---|---|
| Test 1: Rule Compliance | Eligible (`layak`) | 20 | Meets all 4 criteria (Income $\leq$ 2M, Dependents $\geq$ 2, Assets $\leq$ 15M, Decile $\leq$ 3). |
| | Ineligible (`tidak_layak`) | 20 | 15 single-criterion failures and 5 multi-criteria failures. |
| | Needs Review (`perlu_ditinjau`) | 20 | 10 cases with missing mandatory fields; 10 cases with Rp 0 income. |
| Test 2: Boundary & Duplication | Boundary Value Analysis | 12 | 3 points per criterion (below, at, and above limit) across 4 criteria. |
| | Duplicate & Post-Edit Re-run | 2 | Second verification attempt before and after updating citizen income. |
| Test 3: Traceability Protocol | Officer Audit Protocol | 10 | Designed protocol; unexecuted due to synthetic evaluation scope. |

**Table 8.** Consolidated Output (2): Evaluation metrics per experimental condition

| Condition | Cases ($N$) | Status Match | Reason Match | False Acc/Rej | Auto-Decided | Observed System Behavior |
|---|---|---|---|---|---|---|
| Test 1: Eligible | 20 | 20 (100%) | 20 (100%) | 0 / 0 | — | All 4 criteria logged as passed in `detail_pengecekan`. |
| Test 1: Ineligible | 20 | 20 (100%) | 20 (100%) | 0 / 0 | — | Failed criteria explicitly identified and reported. |
| Test 1: Missing Data | 10 | 10 (100%) | 10 (100%) | 0 / 0 | 0 (0%) | Successfully routed to `perlu_ditinjau` (missing fields). |
| Test 1: Rp 0 Anomaly | 10 | 10 (100%) | 10 (100%) | 0 / 0 | 0 (0%) | Successfully routed to `perlu_ditinjau` (field survey flagged). |
| Test 2: Boundary Values | 12 | 12 (100%) | 12 (100%) | 0 / 0 | — | Operators ($\leq$, $\geq$) correctly evaluated at exact limits. |
| Test 2: Duplicate Run | 1 | Compliant | Compliant | — | — | Blocked via `AlreadyVerifiedError`; database intact. |
| Test 2: Post-Correction | 1 | Blocked | — | — | — | Re-run blocked by immutability check despite data edit. |
| Test 3: Officer Traceability | 10 | Protocol Plan | Protocol Plan | — | — | Human trial unexecuted; evaluation protocol documented. |

## 5 Discussion: Citizen Decile Appeal Workflow and Re-Evaluation Bottlenecks

To contextualize the data correction bottleneck identified in Test 2 (TC-DUP-02) and Table 9, this section examines Sociovault's administrative appeal mechanism. Sociovault incorporates a citizen-facing decile appeal module (`/pengajuan-desil`) designed to enable participatory data maintenance while safeguarding auditability.

### 5.1 Decile Appeal Architecture and Procedure

The appeal workflow operates across three sequential steps:

1. **Status Inspection and Appeal Initiation (Fig. 2):** Citizens access their personal dashboard (`/dashboard`) to review their current demographic record, welfare decile classification, and program history. When discrepancies arise, citizens initiate an appeal via "*Ajukan Perubahan Desil.*"
2. **Socioeconomic Evidence Submission (Fig. 3):** Citizens submit updated factual attributes (number of dependents, declared monthly income, narrative justification) alongside verifiable physical evidence, such as photographic proof of housing conditions and poverty certificates (SKTM).
3. **Administrative Audit and Verification Queue (Fig. 4):** Submitted appeals enter an administrative queue (`/riwayat-pengajuan-desil`) labeled "*Menunggu Verifikasi Petugas.*" Authorized village officers examine the submitted evidence, conduct on-site verification if warranted, and approve or reject the request with mandatory review notes.

### 5.2 Architectural Decoupling: Profile Maintenance vs. Program Immutability

While the decile appeal workflow successfully enables citizens and village officers to update master demographic records in the `warga` table (e.g., updating `warga.desil` and `warga.penghasilan`), it reveals an important architectural separation from the eligibility verification engine:

**Table 9.** Consolidated Output (3): Representative failure, anomaly, and limitation cases

| Case ID | Input Condition | System Output | Issue Category | Root Cause and Traceability Analysis |
|---|---|---|---|---|
| TC-36 | Income Rp 3M (> limit), 1 dependent (< limit) | `tidak_layak` | Expected Failure | Multi-criteria failure; `detail_pengecekan` explicitly recorded both failing attributes. |
| TC-41 | Mandatory `penghasilan` is NULL | `perlu_ditinjau` | Incomplete Data | Missing mandatory attribute halted automated rejection, deferring to administrative review. |
| TC-51 | Income Rp 0 with 3 dependents | `perlu_ditinjau` | Economic Anomaly | Zero income treated as an anomaly requiring physical field survey rather than immediate eligibility. |
| TC-DUP-02 | Profile income updated (2.5M → 1.5M), re-verified | Rejected (`AlreadyVerified`) | Architecture Boundary | Single-verification constraint per citizen–program lacks automated re-evaluation path. |
| DEC-EST | Citizen lacks official decile (auto-calculated) | Decile estimated heuristically | Simulation Heuristic | Per-capita income/asset formula provides prototype estimates but lacks empirical policy validation. |

**Table 10.** Consolidated Output (4): Alignment between empirical results and research questions

| Research Question | Empirical Evidence | Evaluation and Limitations |
|---|---|---|
| **RQ1:** Adherence to criteria without premature decisions on incomplete cases | Test 1 achieved 100% label and rationale compliance across 60 cases; 0% premature decisions on review cases. Test 2 verified exact boundary behavior ($\leq, \geq$). | **Positive adherence, bounded scope:** The deterministic engine executes coded rules with 100% compliance. However, adherence to synthetic rules demonstrates code correctness rather than social targeting efficacy or policy fairness. |
| **RQ2:** Traceability of rationales and post-correction re-evaluation | `detail_pengecekan` logged 100% of criteria rationales. In Test 2 (TC-DUP-02), re-verification was rejected after profile updates. | **Traceable rationale, decoupled re-evaluation:** Decisions are fully traceable at the per-criterion level. However, updating citizen profiles does not permit re-verification under the same program due to strict decision immutability. |

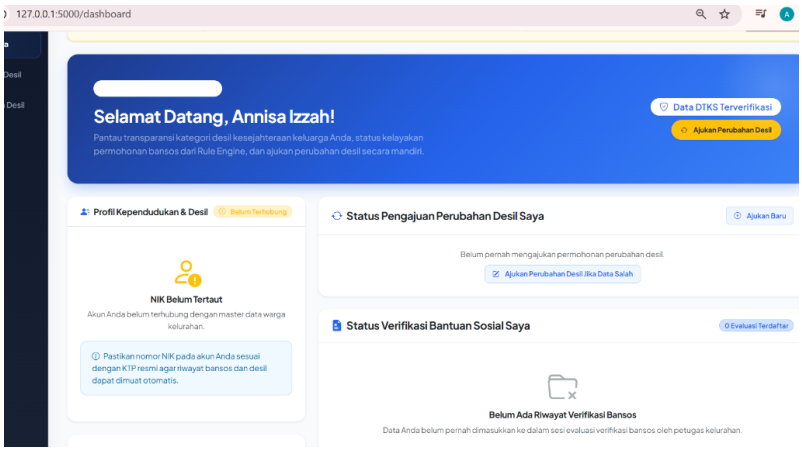



**Fig. 2.** Citizen dashboard displaying registered welfare decile and appeal trigger.

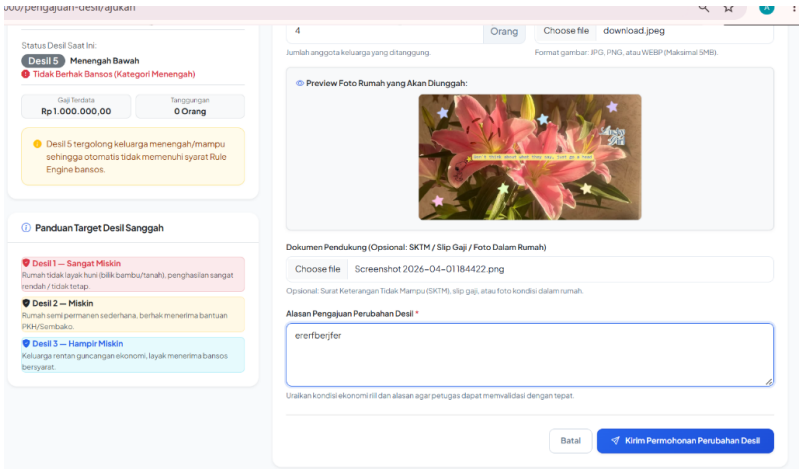



**Fig. 3.** Decile appeal interface with comparative guidance and photographic evidence submission.

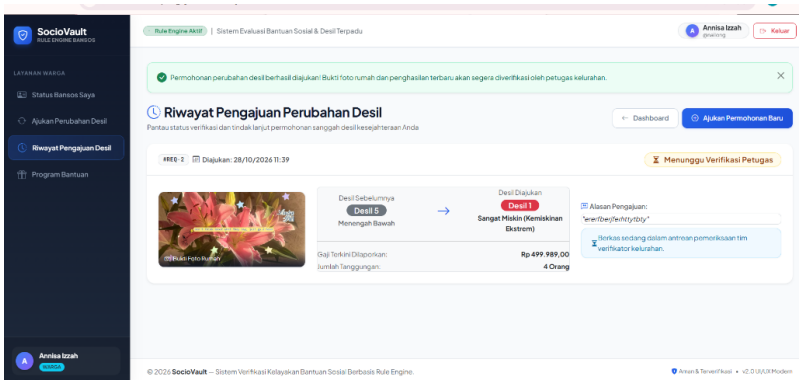



**Fig. 4.** Appeal audit tracking showing submitted socioeconomic evidence and officer review status.

- *Master Profile Updating:* When an officer approves an appeal, the citizen's profile attributes are updated in the database with full audit logging (`PengajuanDesil`).
- *Engine Decision Immutability:* In `app/verifikasi/engine.py`, the rule engine enforces strict single-verification immutability to prevent duplicate disbursements for a given program cycle. Because prior verification records (`Verifikasi`) remain committed, running verification again for the same citizen and program pair triggers `AlreadyVerifiedError`.

This architectural separation explains the outcome of test case TC-DUP-02. The decile appeal workflow resolves data correction at the demographic registry level, but it does not automatically invalidate or recalculate existing program determinations. In municipal administration, this separation prevents unauthorized tampering with past disbursement rolls. However, to support legitimate re-examinations following verified economic changes, the system requires a formal, auditable re-evaluation protocol (such as administrative status revocation or multi-round verification lifecycles) rather than silent re-runs.

## 6 Conclusion

Based on the empirical evaluation of the Sociovault prototype, the findings are structured into four core conclusions:

1. **Deterministic Rule Compliance (RQ1):** Across 60 synthetic cases spanning eligible, ineligible, and inconclusive profiles, Sociovault's multi-stage rule engine achieved 100.0% label compliance and 100.0% rationale compliance against defined program criteria. The system produced zero false acceptances, zero false rejections, and zero premature automated determinations on review cases, correctly delegating ambiguous cases to administrative officers.

2. **Traceability and Re-Evaluation Realities (RQ2):** The prototype provides complete per-criterion decision traceability via `detail_pengecekan`, allowing officers to audit the exact criteria causing eligibility or review statuses. However, a critical architectural limitation exists regarding post-decision re-evaluation: while the decile appeal workflow enables citizens and officers to correct demographic profiles, the verification engine enforces strict single-verification immutability to prevent duplicate claims. Consequently, updated citizen profiles cannot be automatically re-evaluated under the same program without a formal administrative reset mechanism.
3. **Evaluation Boundaries:** These conclusions are subject to specific experimental constraints: evaluations were conducted using synthetic test cases for a single assistance program with simulation thresholds; official institutional data was not utilized; and the officer traceability protocol (Test 3) represents an evaluation design that has not yet been deployed in field trials.
4. **Practical Contribution and Future Work:** Sociovault demonstrates that multi-stage rule engines combined with granular audit logging provide a transparent, accountable framework for municipal welfare assessment. Future work should focus on integrating formal decision-revocation lifecycles to support auditable re-evaluations, validating heuristic decile formulas against official poverty registries, and conducting empirical field studies with public assistance caseworkers.

## References

1. Alatas, V., Banerjee, A., Hanna, R., Olken, B.A., Julia, M.: Targeting the poor: Evidence from a field experiment in Indonesia. American Economic Review **102**(4), 1206–1240 (2012). https://doi.org/10.1257/aer.102.4.1206
2. Department of Social Welfare and Development (DSWD): Guidelines on the enhanced deduplication procedure amending memorandum circulars no. 14 and no. 17, series of 2020. DSWD Memorandum Circular No. 18, Series of 2020, Quezon City, 22 Juli 2020 (2020)
3. Hanna, R., Olken, B.A.: Universal basic incomes versus targeted transfers: Anti-poverty programs in developing countries. Journal of Economic Perspectives **32**(4), 201–226 (2018). https://doi.org/10.1257/jep.32.4.201
4. OpenG2P: Deduplication service documentation, versi 1.0. https://github.com/newlogic/openg2p/blob/openg2p-ng-from-newlogic-g2p/docs/deduplicator/index.md (2024), diakses: 2024
5. openIMIS Initiative: Advanced deduplication algorithm (SR-053), dokumen kebutuhan sistem. Tech. rep., openIMIS Initiative (Januari 2022), https://openimis.atlassian.net/wiki/spaces/OP/pages/4129980470
6. Owino, B.: Harmonising data systems for cash transfer programming in emergencies in Somalia. Journal of International Humanitarian Action **5**(1), 11 (2020). https://doi.org/10.1186/s41018-020-00077-1, https://doi.org/10.1186/s41018-020-00077-1
7. Pallets Projects: Flask documentation, versi 3.0. https://flask.palletsprojects.com/ (2024), diakses: 2024

8. Presiden Republik Indonesia: Undang-undang nomor 11 tahun 2009 tentang kesejahteraan sosial. Lembaran Negara Republik Indonesia Tahun 2009 Nomor 12, Tambahan Lembaran Negara Republik Indonesia Nomor 4967, Pasal 5 dan Pasal 34 (2009)
9. Presiden Republik Indonesia: Peraturan presiden nomor 95 tahun 2018 tentang sistem pemerintahan berbasis elektronik. Lembaran Negara Republik Indonesia Tahun 2018 Nomor 182 (2018)
10. Presiden Republik Indonesia: Undang-undang nomor 27 tahun 2022 tentang pelindungan data pribadi. Lembaran Negara Republik Indonesia Tahun 2022 Nomor 196, Tambahan Lembaran Negara Republik Indonesia Nomor 6820 (2022)
11. Progress Software: Eligibility decisions at scale: How to stay fair, consistent and audit ready. White Paper, https://www.progress.com/ (2023), diakses: 2024
12. SQLAlchemy: Sqlalchemy documentation, versi 2.0. https://www.sqlalchemy.org/ (2024), diakses: 2024
13. State of California, Department of Health Services: All county welfare directors letter no. 87-15: Name/birthdate match process. Sacramento, CA, 14 Mei 1987 (1987), https://www.dhcs.ca.gov/services/medical/eligibility/letters/Documents/c87-15.pdf
14. State of California, Department of Health Services: All county welfare directors letter no. 94-27. Sacramento, CA, 15 Agustus 1994 (1994), https://www.dhcs.ca.gov/services/medical/eligibility/letters/Documents/c94-27.pdf
15. Sunny, A.: A Neuro-Symbolic Framework for Accountability in Public-Sector AI. Master's thesis, Department of Computer Science, University of Maryland (2025), https://drum.lib.umd.edu/items/63f229d5-2f3e-45b3-bc14-28a48cad58a9
16. Sunny, A., et al.: A neuro-symbolic framework for legal accountability in public-sector AI. Tech. Rep. Tech. Rep., University of Maryland, College Park, MD (2024), https://dgi.umd.edu/node/15089
17. U.S. General Accounting Office (GAO): Duplicate welfare payments in New York jurisdictions. Tech. Rep. HRD-78-133, U.S. General Accounting Office, Washington, D.C. (1978)
18. Wei, M., Smith, J., Johnson, R.: Envisioning a human-AI collaborative system to transform policies into decision models. arXiv preprint arXiv:2212.06882 (2022), https://arxiv.org/abs/2212.06882